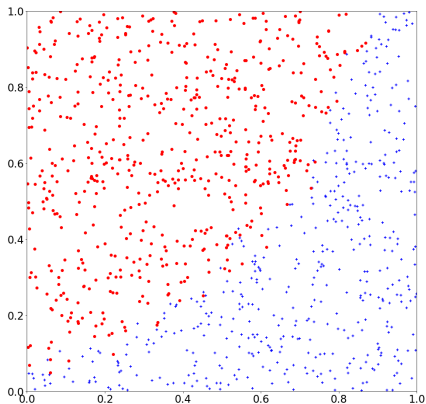


# Foundational Machine Learning: Regularisation

Wenbin Li  
w.li@bath.ac.uk

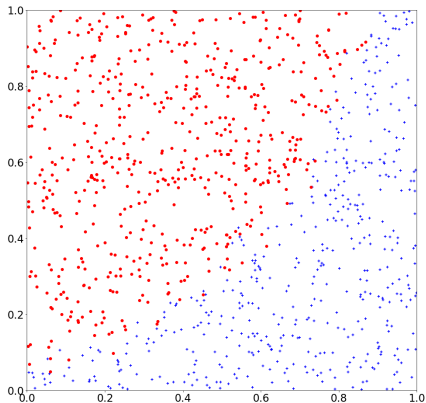


## Underfitting & overfitting

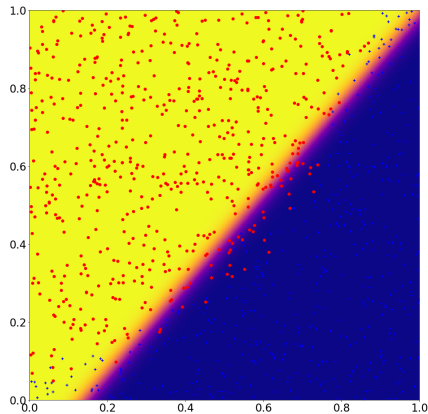


- Curved
- Classes overlap
- How would you divide them?

## Underfitting & overfitting

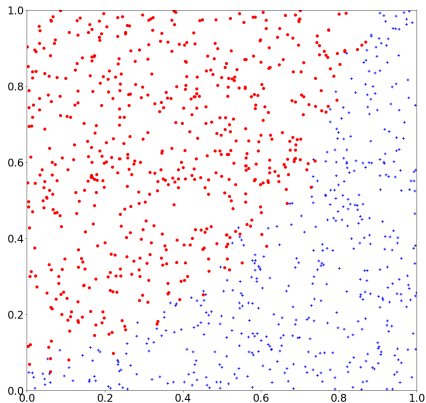


- Curved
- Classes overlap
- How would you divide them?

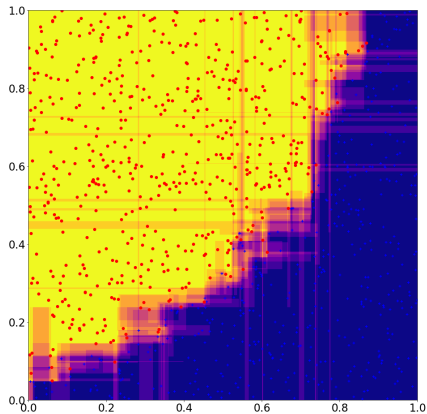


- Underfitting

## Underfitting & overfitting

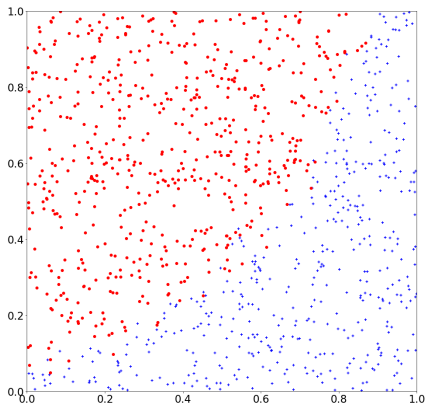


- Curved
- Classes overlap
- How would you divide them?

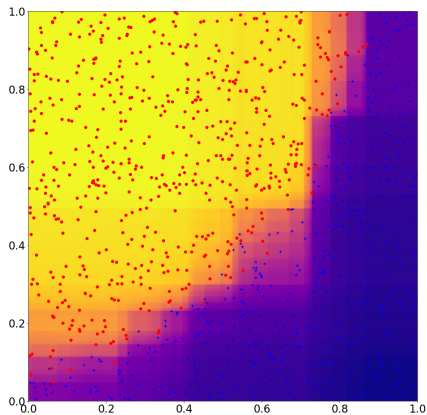


- Overfitting

## Underfitting & overfitting

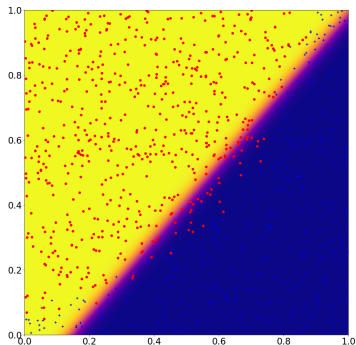


- Curved
- Classes overlap
- How would you divide them?

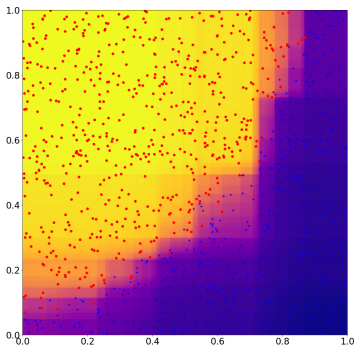


- Balanced

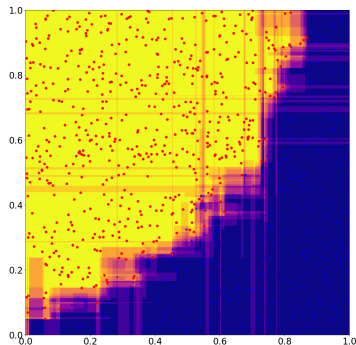
## Underfitting & overfitting



- Underfitting
- Logistic regression



- Balanced
- Tuned random forest
- (scikit learn,  
min\_impurity\_decrease=0.008,  
n\_estimators=512)



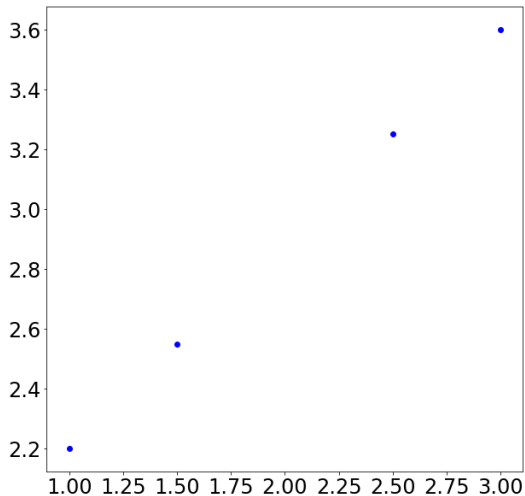
- Overfitting
- Badly tuned random forest
- (scikit learn,  
default parameters)

# Regularisation

- This lecture is about **regularisation**. . .  
    . . . which avoids overfitting (among other things)
- Effectively “extra information”

## Extra information

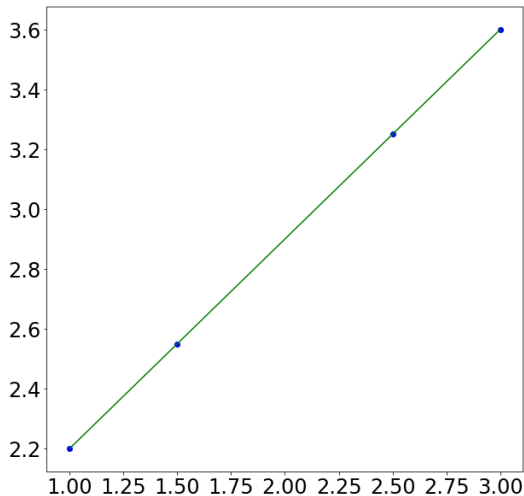
- 1D regression, 4 points





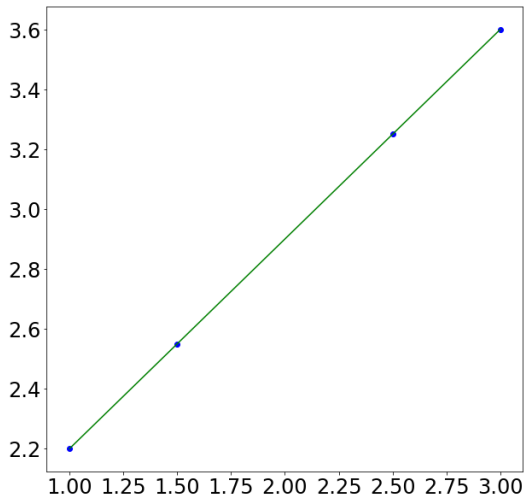
## Extra information

- 1D regression, 4 points
- Linear solution obvious – to us!



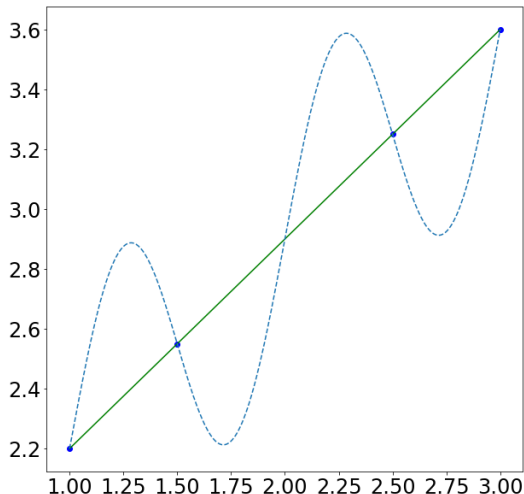
## Extra information

- 1D regression, 4 points
- Linear solution obvious – to us!
- Assume general model – anything goes



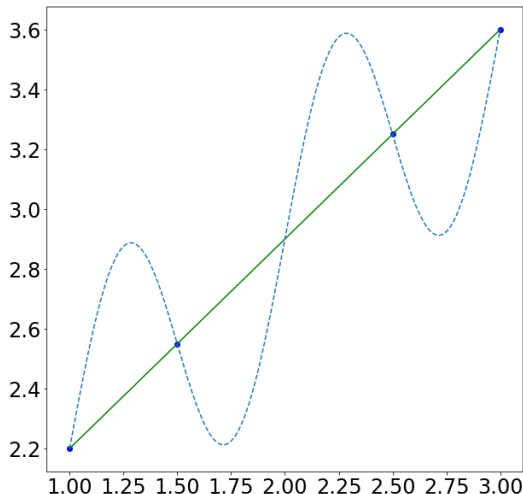
## Extra information

- 1D regression, 4 points
- Linear solution obvious – to us!
- Assume general model – anything goes
- e.g. a sine curve



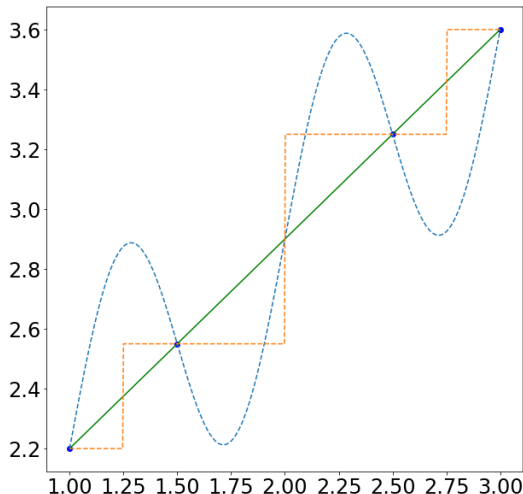
## Extra information

- 1D regression, 4 points
- Linear solution obvious – to us!
- Assume general model – anything goes
- e.g. a sine curve
- Perfect match at known points...
- **Identical cost to a straight line!**



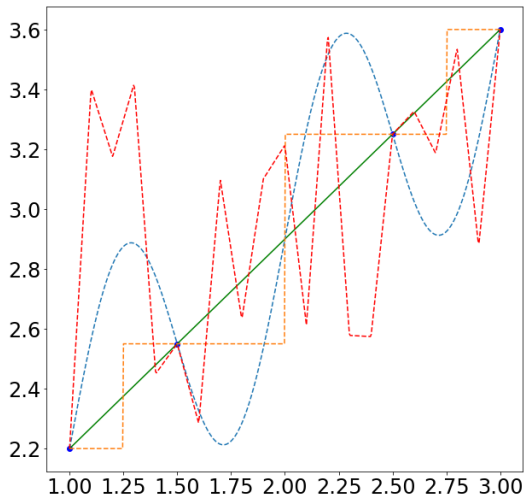
## Extra information

- 1D regression, 4 points
- Linear solution obvious – to us!
- Assume general model – anything goes
- e.g. a sine curve
- Perfect match at known points...
- **Identical cost to a straight line!**



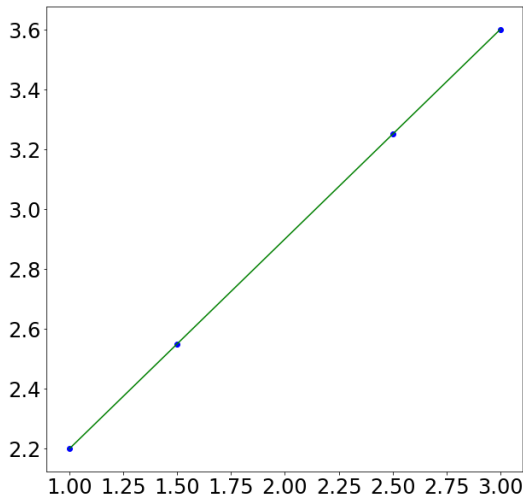
## Extra information

- 1D regression, 4 points
- Linear solution obvious – to us!
- Assume general model – anything goes
- e.g. a sine curve
- Perfect match at known points...
- **Identical cost to a straight line!**



## Extra information

- 1D regression, 4 points
- Linear solution obvious – to us!
- Assume general model – anything goes
- e.g. a sine curve
- Perfect match at known points...
- **Identical cost to a straight line!**
- Regularisation emphasises **simpler model**  
(straight line)
- **Common sense** for models!



The simplest explanation is usually the correct one

- Traceable to Aristotle (384–322 BC)
- Ockham's version: "Plurality must never be posited without necessity"  
(translated from Latin — William of Ockham = 13th century priest)
- Overfitting: Unjustifiably complex explanation



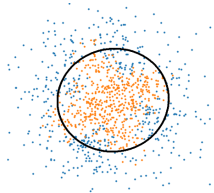
## Why regularise?

- Overfitting
- Ill posed problem
- Human understanding
- Easier optimisation

Often several at once

## Reason: Overfitting

- Already seen...
- Overfitting = Fitting to noise
- Another example (SVM, rbf kernel – will be taught later):

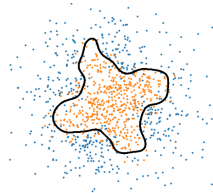


$$\gamma = 0.01, C = 1$$

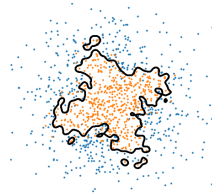


$$\gamma = 0.1, C = 1$$

(about right)



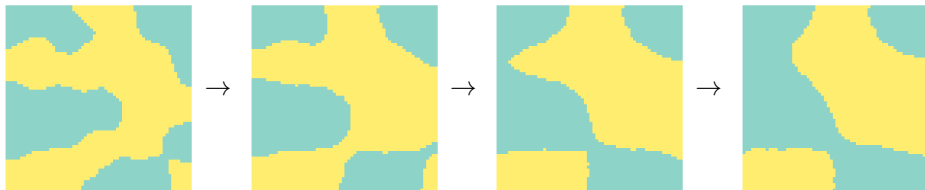
$$\gamma = 1.0, C = 1$$



$$\gamma = 10.0, C = 1$$

## Reason: Ill posed

- Ill posed: Multiple equally good solutions (line from earlier)
- e.g. order irrelevant: bricks when making a wall, nodes in a neural network
- Regularisation: Force selection, even if arbitrary
- Without: Optimisation can “drift” between solutions:



(colours represent different solutions, chasing each other around an image)

- Drifting = never converges, all solutions bad

## Reason: Human understanding

- Goal: Learn  $y = f_{\theta}(x)$
- Alternatively: Learn  $y = f_{\theta}(z)$  and  $z = f_{\eta}(x)$   
Subject to  $z$  being useful in some way, i.e. human interpretable

## Reason: Human understanding

- Goal: Learn  $y = f_{\theta}(x)$
- Alternatively: Learn  $y = f_{\theta}(z)$  and  $z = f_{\eta}(x)$   
Subject to  $z$  being useful in some way, i.e. human interpretable
- Attribute learning:
  - $z = f_{\eta}(x)$  encodes: Has tail, black & white, four legs etc.
  - $x = f_{\theta}(z)$  encodes: Is zebra, is horse, is penguin

## Reason: Human understanding

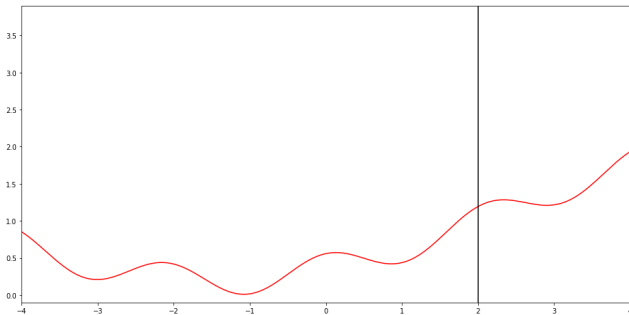
- Goal: Learn  $y = f_{\theta}(x)$
- Alternatively: Learn  $y = f_{\theta}(z)$  and  $z = f_{\eta}(x)$   
Subject to  $z$  being useful in some way, i.e. human interpretable
- Attribute learning:
  - $z = f_{\eta}(x)$  encodes: Has tail, black & white, four legs etc.
  - $x = f_{\theta}(z)$  encodes: Is zebra, is horse, is penguin
- Regularisation towards simpler model *as judged by a human*

## Reason: Easier optimisation

- “Drifting” between solutions already an example
- Regularisation: Smooths cost function  $\rightarrow$  fewer local minima  
(also removes stationary points, accelerating convergence)

## Reason: Easier optimisation

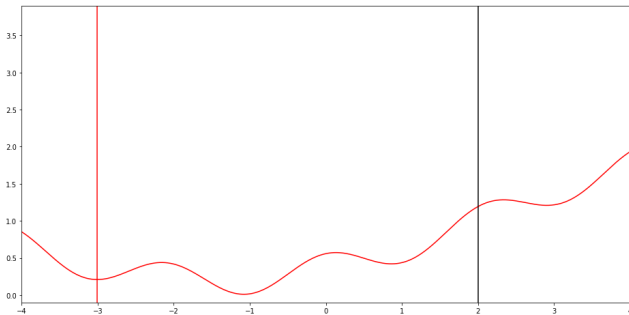
- “Drifting” between solutions already an example
- Regularisation: Smooths cost function  $\rightarrow$  fewer local minima  
(also removes stationary points, accelerating convergence)
- Find minima of red function  
(starting at black vertical,  
global optima is  $x = 1$ )





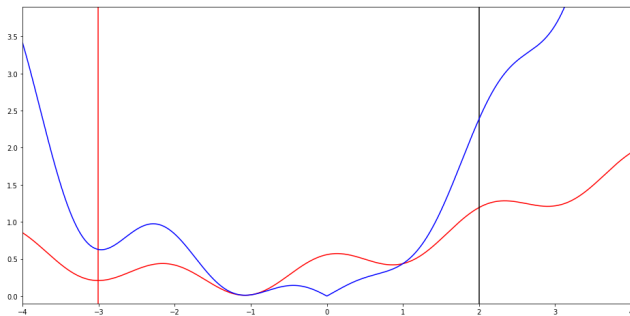
## Reason: Easier optimisation

- “Drifting” between solutions already an example
- Regularisation: Smooths cost function  $\rightarrow$  fewer local minima  
(also removes stationary points, accelerating convergence)
- Find minima of red function  
(starting at black vertical, global optima is  $x = 1$ )
- Stuck at  $x = 3$   
(using BFGS)



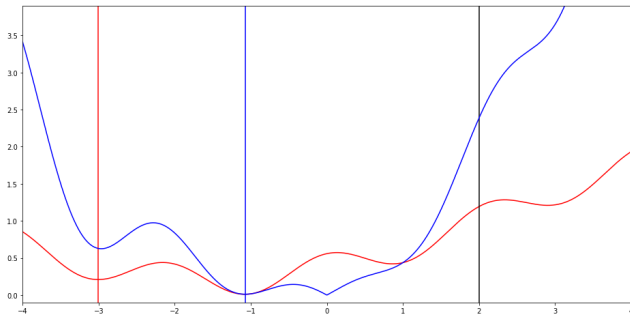
## Reason: Easier optimisation

- “Drifting” between solutions already an example
- Regularisation: Smooths cost function  $\rightarrow$  fewer local minima  
(also removes stationary points, accelerating convergence)
- Find minima of red function  
(starting at black vertical, global optima is  $x = 1$ )
- Stuck at  $x = 3$   
(using BFGS)
- Blue: L1 regularisation  
(pushing answer towards  $x = 0$ )



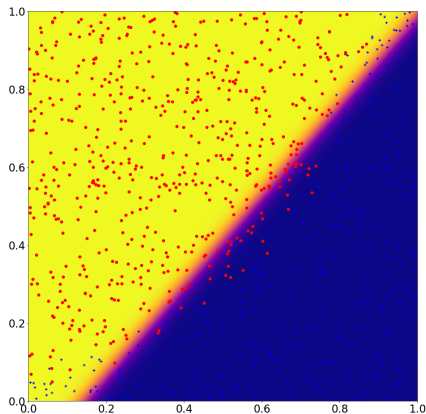
## Reason: Easier optimisation

- “Drifting” between solutions already an example
- Regularisation: Smooths cost function  $\rightarrow$  fewer local minima  
(also removes stationary points, accelerating convergence)
- Find minima of red function  
(starting at black vertical, global optima is  $x = 1$ )
- Stuck at  $x = 3$   
(using BFGS)
- Blue: L1 regularisation  
(pushing answer towards  $x = 0$ )
- Finds better optima  
(happens to be global)



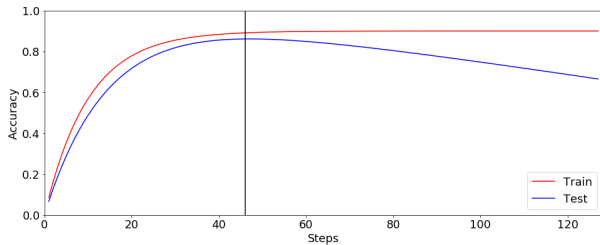
## Aside: Model limits

- Model limits  $\approx$  regularisation  
e.g. Logistic regression only does straight lines
- But rarely the “right amount”,  
and no hyperparameter to tune



## Aside: Early stopping

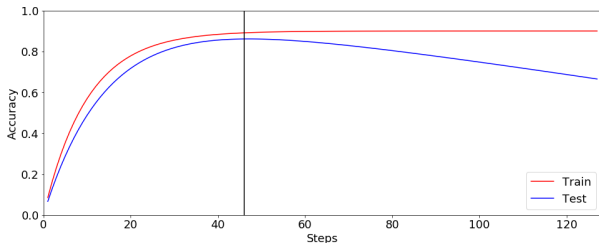
- Model starts simple...  
...gets more complicated as optimisation runs...  
...until overfitting



- $\therefore$  stop early!

## Aside: Early stopping

- Model starts simple...  
...gets more complicated as optimisation runs...  
...until overfitting



- $\therefore$  stop early!
- Bad hack — avoid  $\implies$  Regularisation too weak - strengthen!
- May still be least terrible solution (common for neural networks)

## Aside: Quantity I

- More data  $\implies$  less regularisation required
- Infinite data  $\implies$  no regularisation!  
(simple lookup – nearest neighbour)

## Aside: Quantity I

- More data  $\implies$  less regularisation required
- Infinite data  $\implies$  no regularisation!  
(simple lookup – nearest neighbour)
- Hyperparameters control regularisation strength
- More/less data  $\rightarrow$  different hyperparameter values  
(can still tune with subset of train then fine tune with all)



## Aside: Quantity I

- More data  $\implies$  less regularisation required
- Infinite data  $\implies$  no regularisation!  
(simple lookup – nearest neighbour)
- Hyperparameters control regularisation strength
- More/less data  $\rightarrow$  different hyperparameter values  
(can still tune with subset of train then fine tune with all)
- Models often have sweet spot:
  - Not enough data  $\rightarrow$  fail
  - Too much data  $\rightarrow$  stop improving (underfit)

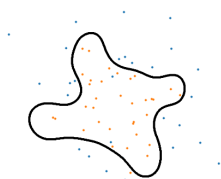
## Aside: Quantity II



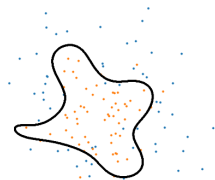
exemplars = 16,  $\gamma = 0.5$ ,  
accuracy = 83.5%



exemplars = 32,  $\gamma = 0.5$ ,  
accuracy = 83.4%



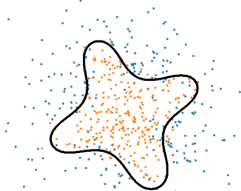
exemplars = 64,  $\gamma = 0.5$ ,  
accuracy = 87.5%



exemplars = 128,  $\gamma = 0.25$ ,  
accuracy = 86.5%



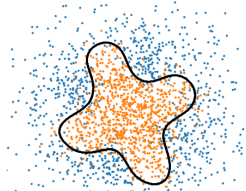
exemplars = 256,  $\gamma = 0.25$ ,  
accuracy = 87.2%



exemplars = 512,  $\gamma = 0.1$ ,  
accuracy = 89.1%



exemplars = 1024,  $\gamma = 0.1$ ,  
accuracy = 89.3%



exemplars = 2048,  $\gamma = 0.25$ ,  
accuracy = 89.3%

## Model kinds I

- Discussed **why**
- **How** depends on model kind...

## Model kinds I

- Discussed **why**
- **How** depends on model kind...
- Non-probabilistic
  - Arbitrary loss functions
- Probabilistic
  - Maximum likelihood (ML) (no regularisation)
  - Maximum a posteriori (MAP)
  - Bayesian

## Non-probabilistic

- Model fitting: Minimises loss function  $L(\theta)$ , e.g. L2:

$$L(\theta) = \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2$$

## Non-probabilistic

- Model fitting: Minimises loss function  $L(\theta)$ , e.g. L2:

$$L(\theta) = \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2$$

- Regularise  $\theta$ , e.g. encourage small parameters:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2 + \lambda \sum_{j=1}^k \theta_k^2$$

- $\lambda$  = regularisation strength — hyperparameter

# Ridge regression I

Also called Tikhonov regression

Define  $f_{\theta}(x_i)$  as:

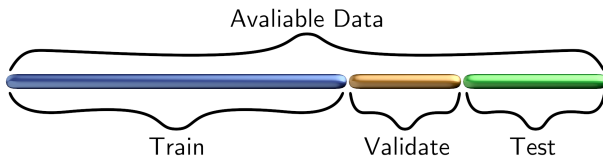
$$y_i = ax_i + b, \quad \theta = [a, b]$$

(linear regression)

Loss function:

$$L(\theta) = \sum_{i=1}^n (y_i - (ax_i + b))^2 + \lambda(a^2 + b^2)$$

## Recap: Measuring performance



- How do we decide on split of dataset?
  - **train** – Give to algorithm to determine the parameters
    - Large training set → Algorithm performs well
  - **validation** – Objective of hyperparameter optimisation
    - Large validation set → Hyperparameter optimisation performs well, to a limit
  - **test** – To report final performance
    - Large test set → Accurate performance estimate, to a limit



## Ridge regression II

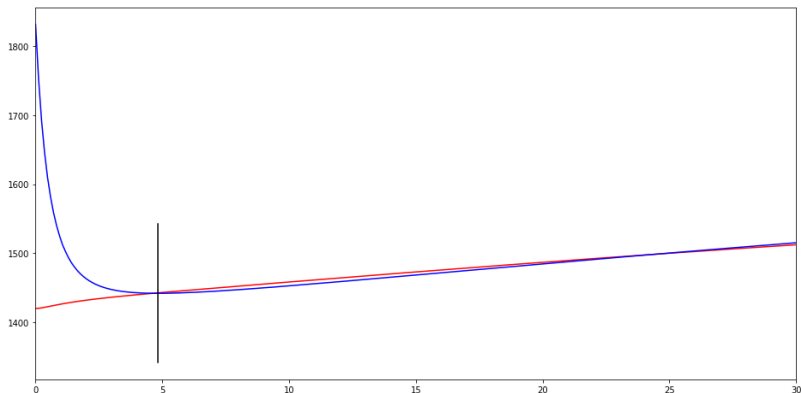
Also called Tikhonov regression

- Estimate diamond price given 9 features (carat, cut, colour, multiple for size)
- Linear model: Train RMSE = 1420; Test RMSE = 1831 (overfit)

## Ridge regression II

Also called Tikhonov regression

- Estimate diamond price given 9 features (carat, cut, colour, multiple for size)
- Linear model: Train RMSE = 1420; Test RMSE = 1831 (overfit)
- Sweep: (x-axis =  $\lambda$ , y-axis = RMSE, blue = validation, red = train)



- Best (black line): Train RMSE = 1443; Validation RMSE = 1442 (test is now validation)

## Lasso, ridge and elastic net

- Ridge regression: (L2 norm, without square root)

$$L(\theta) = \sum_{i=1}^n (y_i - (ax_i + b))^2 + \lambda(a^2 + b^2)$$

## Lasso, ridge and elastic net

- Ridge regression: (L2 norm, without square root)

$$L(\theta) = \sum_{i=1}^n (y_i - (ax_i + b))^2 + \lambda(a^2 + b^2)$$

- Lasso regression: (L1 norm)

$$L(\theta) = \sum_{i=1}^n (y_i - (ax_i + b))^2 + \lambda(|a| + |b|)$$

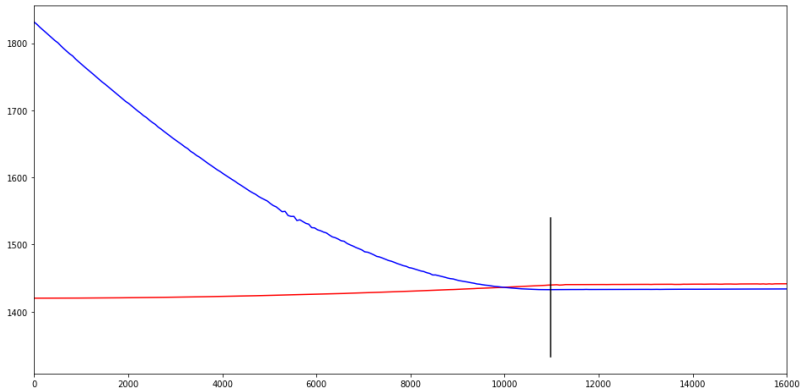
- Elastic net regression: (blend of lasso and ridge)

$$L(\theta) = \sum_{i=1}^n (y_i - (ax_i + b))^2 + \lambda (\gamma(|a| + |b|) + (1 - \gamma)(a^2 + b^2))$$

(two hyperparameters:  $\lambda$  and  $\gamma$ )

## Lasso regression

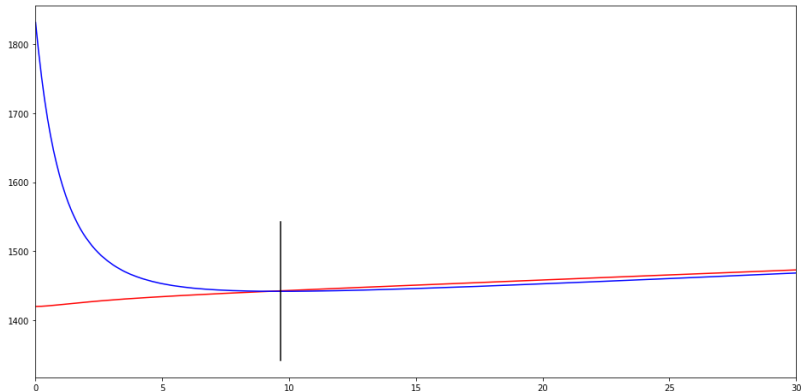
- Sweep: (x-axis =  $\lambda$ , y-axis = RMSE, blue = validation, red = train)



- Best (black line): Train RMSE = 1440; Validation RMSE = 1433
- Increasing hyperparameter leads to sparser coefficients

## Elastic net regression

- $\gamma = 0.5$  (good default if not sweeping)
- Sweep: (x-axis =  $\lambda$ , y-axis = RMSE, blue = validation, red = train)



- Best (black line): Train RMSE = 1443; Validation RMSE = 1442

## Robustness

- Results (RMSE):
  - Linear: 1831
  - Ridge: 1442
  - Lasso: 1433
  - Elastic: 1442
- Little difference!

- Results (RMSE):
  - Linear: 1831
  - Ridge: 1442
  - Lasso: 1433
  - Elastic: 1442
- Little difference!
- More complicated problems: L1 (lasso) often better than L2 (ridge)  
(L1 better at ignoring irrelevant features)
- Elastic-net lets hyperparameter optimisation decide
- There are more complex regularisers, e.g. robust statistics



## Summary

- Regularisation embodies common sense
  - Smooth the model on noisy data
  - Help to stop the training earlier
  - Avoid overfitting
- Models can be probabilistic or not
- Training and validating are required

## Further reading

- Chapter 28, of “Information Theory, Inference, and Learning Algorithms” by MacKay.